

FLASC: Aprendizado Federado com Agrupamento Suave Adaptativo para Mitigação de Heterogeneidade de Dados

Pedro Gabriel do Valle Nogueira¹

¹ Programa de Pós-Graduação em Engenharia Eletrônica
Universidade do Estado do Rio de Janeiro (UERJ) – Rio de Janeiro, RJ – Brasil

Resumo.

Abstract.

1. Introdução

A crescente digitalização de serviços e a onipresença de dispositivos computacionais pessoais gera um volume de dados distribuídos sem precedentes, o que impulsiona o desenvolvimento de soluções baseadas em aprendizado de máquina (ML). Tradicionalmente, uma solução de ML exige que os dados estejam em um único dispositivo. No entanto, os dados distribuídos geralmente devem permanecer nos seus dispositivos de origem por questões de privacidade. Para superar esse obstáculo, o Aprendizado Federado (do inglês, *Federated Learning* – FL) surgiu como um paradigma onde um servidor mantém um modelo global e distribui cópias para clientes selecionados na rede. Nessa arquitetura, os dispositivos realizam o treinamento local com seus próprios dados e transmitem apenas as atualizações dos parâmetros. O servidor então agrega essas contribuições via média ponderada (FedAvg) para atualizar o modelo global. Todo esse processo se repete por diversas rodadas visando a convergência [1, 2].

Apesar dos avanços que o FL proporcionou para sistemas de recomendação e para a área da saúde, sua eficácia é frequentemente desafiada pela heterogeneidade estatística dos dados nos clientes (dados não-IID). Essa heterogeneidade impacta negativamente a agregação, tornando o modelo global subótimo para a maioria dos participantes. Com o objetivo de solucionar esse problema, uma linha de pesquisa proeminente foca no Aprendizado Federado Clusterizado (do inglês, *Clustered Federated Learning* – CFL). A intuição por trás do CFL é agrupar clientes com distribuições de dados semelhantes e treinar modelos especializados para cada grupo. Essa abordagem tem demonstrado melhorar significativamente o desempenho em ambientes heterogêneos.

precisa ser determinado com o algoritmo. Briggs et al. [3] propõe uma estratégia de CFL que tenta determinar esse número com o algoritmo de agrupamento hierárquico. O problema é que o agrupamento hierárquico gera grupos disjuntos, e é natural que um cliente possa ter dados que se encaixem em múltiplos grupos. Outro ponto é que o agrupamento acontece em uma rodada arbitrária. Se o agrupamento for subótimo, isso impacta todo o restante do treinamento.

Outras estratégias de CFL assumem um número de agrupamentos conhecido.

No IFCA, proposto por Ghosh et al. [4], o servidor mantém múltiplos modelos globais e os transmite aos clientes. Cada participante avalia os modelos recebidos com seus próprios dados e seleciona aquele que possui menor perda como base para

o treinamento. Terminado o treinamento, ele envia o modelo treinado de volta para o servidor junto do identificador de qual modelo usou. No servidor, cada modelo global é atualizado somente com os clientes que os utilizaram para treinar. O IFCA exige que o cliente se vincule estritamente a um grupo por rodada, o que limita a possibilidade de usar conhecimento de outros grupos localmente. Isso também limita contribuição do cliente na atualização do modelos globais, já que só o modelo do grupo que ele escolheu vai se beneficiar.

Li et al. [2] apresenta o Aprendizado Federado com Agrupamento Suave (FLSC) como uma melhoria do IFCA. Essa melhoria possibilitou aos clientes pertencerem a múltiplos grupos simultaneamente. Graças a isso, o FLSC demonstrou capturar melhor a natureza dos dados não-IID em relação ao seu antecessor [?]. Contudo, o método impõe um número fixo de grupos para todos os clientes participarem, e o peso de participação do cliente em qualquer grupo é sempre igual. Isso ainda não reflete perfeitamente um cenário real. Por exemplo, num sistema de recomendação, conteúdos do tipo A e B podem estar num cliente c1, enquanto outro cliente c2 tem conteúdos do tipo B, C, D e E. É natural pensar que c1 participe de menos grupos de recomendação que c2. Além disso, não necessariamente todos conteúdos vão ser recomendados na mesma frequência. Pode ser que c2 consuma muito conteúdo do tipo B, consequentemente gerando muito mais dados do tipo B. Logo faz sentido pensar que c2 participe bem mais do grupo que inclui conteúdo do tipo B.

Visando superar as limitações do FLSC, este trabalho propõe o FLASC (*Federated Learning with Adaptive Soft Clustering*). Diferente do FLSC, o FLASC permite não só que a quantidade de grupos que um cliente participe se ajuste conforme a sua distribuição local, como ele também pondera o peso de participação do cliente em cada grupo. Experimentos ...

O restante deste trabalho está organizado da seguinte forma: a Seção 2 descreve detalhadamente o funcionamento do algoritmo FLASC; a Seção 3 apresenta a metodologia de avaliação e o ambiente experimental; a Seção ?? expõe e discute os resultados obtidos; e, finalmente, a Seção ?? traz as considerações finais e propostas de trabalhos futuros.

2. FLASC

O Algoritmo 1 detalha o fluxo operacional completo do FLASC.

Algoritmo 1: Algoritmo FLASC

Input: Parâmetros iniciais K , tolerância ρ , sensibilidade ϵ

Data: Conjuntos de dados locais $\{\mathcal{D}_c\}$ nos clientes

```
1 Servidor: Inicializa  $\mathcal{W}_1 = \{W_{1,1}, \dots, W_{K,1}\};$ 
2 for cada rodada  $t = 1, \dots, T$  do
3   Servidor seleciona aleatoriamente um subconjunto de clientes  $P_t$ ;
4   Servidor transmite  $\mathcal{W}_t$  para todos os clientes em  $P_t$ ;
5   Cada cliente  $c \in P_t$  em paralelo:
6     Calcula perdas locais  $L_{c,i} = \text{EntropiaCruzada}(W_{i,t}, \mathcal{D}_c)$  para
        $i \in \{1, \dots, K\};$ 
7      $L_{\min} \leftarrow \min_i \{L_{c,i}\}, L_{\max} \leftarrow \max_i \{L_{c,i}\};$ 
8     if  $L_{\max} - L_{\min} \leq \epsilon$  then
9        $i' \sim \text{Unif}\{1, \dots, K\}, S_c \leftarrow \{i'\};$ 
10       $\alpha_{c,i'} \leftarrow 1;$ 
11    else
12       $\hat{L}_{c,i} \leftarrow \frac{L_{c,i} - L_{\min}}{L_{\max} - L_{\min}}$  para cada  $i \in \{1, \dots, K\};$ 
13       $S_c \leftarrow \{i \mid \hat{L}_{c,i} \leq \rho\};$ 
14       $\alpha_{c,i} \leftarrow \frac{1 - \hat{L}_{c,i}}{\sum_{j \in S_c} (1 - \hat{L}_{c,j})}$  para cada  $i \in S_c;$ 
15    end
16     $W_{c,t} \leftarrow \sum_{i \in S_c} \alpha_{c,i} W_{i,t};$ 
17     $W_{c,t}^{\text{end}} \leftarrow \text{TreinamentoLocal}(W_{c,t}, \mathcal{D}_c);$ 
18    Envia  $\{W_{c,t}^{\text{end}}, S_c\}$  para o servidor;
19  Servidor:
20  for cada cluster  $i \in \{1, \dots, K\}$  do
21    if  $\{c \in P_t \mid i \in S_c\} \neq \emptyset$  then
22       $W_{i,t+1} \leftarrow \frac{\sum_{c \in \{j \in P_t \mid i \in S_j\}} |\mathcal{D}_c| W_{c,t}^{\text{end}}}{\sum_{c \in \{j \in P_t \mid i \in S_j\}} |\mathcal{D}_c|};$ 
23    else
24       $W_{i,t+1} \leftarrow W_{i,t};$ 
25    end
26  end
27 end
```

Tanto o FLASC quanto o FLSC gerenciam um conjunto de K modelos globais no servidor, representados por $\mathcal{W}_t = \{W_{1,t}, W_{2,t}, \dots, W_{K,t}\}$. No início de cada rodada de treinamento, o servidor seleciona uma fração de clientes para participação e realiza a transmissão de $\mathcal{W}_t$. Uma vez recebidos os modelos, cada cliente participante c mensura o desempenho de cada modelo calculando a Entropia Cruzada em relação ao seu conjunto de dados local $\mathcal{D}_c$ (Linha 6).

2.1. Seleção dos modelos no cliente

A partir das perdas calculadas, o FLASC avalia a amplitude entre a maior e a menor perda ($L_{max} - L_{min}$). Se essa diferença for inferior ao limiar de sensibilidade ϵ , os modelos são considerados indistinguíveis para o cliente e um único modelo é selecionado aleatoriamente (Linha 9). Caso contrário, as perdas são normalizadas no intervalo $[0, 1]$ (Linha 12).

Em seguida, define-se o conjunto S_c dos índices dos modelos selecionados através de um limiar $\rho \in [0, 1]$ (Linha 13). Este limiar estabelece um teto de tolerância para a perda relativa, assegurando que apenas os modelos com desempenho próximo ao melhor candidato local (L_{min}) componham o conjunto de fusão. Diferentemente do FLSC, onde o tamanho do conjunto de seleção N é fixado a priori, o FLASC permite uma seleção adaptativa, evitando a inclusão de modelos com erros excessivos apenas para satisfazer uma restrição de cardinalidade. Adicionalmente, para cada modelo selecionado, o FLASC atribui um peso de contribuição $\alpha_{c,i}$ que decresce linearmente em função da perda normalizada (Linhas 10 e 14). Este mecanismo garante que modelos com menor erro local exerçam maior influência na fusão, funcionalidade que não é contemplada na formulação original do FLSC.

2.2. Fusão, treinamento e agregação

Concluída a etapa de seleção, o cliente sintetiza o modelo inicial para o treinamento local, $W_{c,t}$, mediante a fusão ponderada de $\mathcal{W}_t$ (Linha 16). Após o treinamento local, os parâmetros atualizados ($W_{c,t}^{end}$) são devolvidos ao servidor acompanhados de S_c (Linhas 17-18). Ao término da rodada, o servidor atualiza os modelos globais de forma independente (Linhas 20-26). Para cada cluster i , a nova versão $W_{i,t+1}$ é obtida por meio de uma média ponderada das atualizações enviadas pelos clientes que integraram o respectivo modelo em sua fusão local. Caso o modelo i não tenha sido selecionado por nenhum cliente na rodada vigente, seus parâmetros permanecem inalterados.

3. Proposta de avaliação

A avaliação do FLASC baseia-se na comparação direta com o FLSC [2], adotado como linha de base para quantificar o impacto da substituição do parâmetro rígido de seleção N pelo limiar adaptativo ρ . Conforme detalhado na Tabela 1, os hiperparâmetros de ambos os algoritmos variam de forma correlacionada para permitir uma comparação em cenários de seleção unitária até inclusão total de modelos ($K = 4$). O hiperparâmetro ϵ do FLASC foi fixado em 1×10^{-5} .

Tabela 1. Mapeamento de hiperparâmetros para comparação entre FLSC e FLASC.

Cenário de Seleção	FLSC (N)	FLASC (ρ)
Mínima	1	0
Intermediária (metade)	2	0,5
Máxima	4	1

Para simular um ambiente de heterogeneidade de dados (*non-IID*), foi utilizado o conjunto de dados CIFAR-10 [5] distribuído via partição de Dirichlet com $\alpha = 0,2$ [6]. Esse cenário reflete redes federadas reais, onde a distribuição de classes é desequilibrada

entre os nós. No total, a simulação compreende 10 clientes participando de 10 rodadas de treinamento, com cada nó reservando 20% de seus dados locais para a etapa de validação.

Quanto à simulação, foi utilizado o framework Flower (v1.24.0) [7] integrado à biblioteca PyTorch [8]. A arquitetura dos modelos consiste em uma rede neural convolucional (CNN) de duas camadas de convolução (6 e 16 filtros de 5×5 , respectivamente) intercaladas por *max-pooling*, seguidas por três camadas densas. O treinamento foi conduzido com otimizador SGD (momento 0,9), taxa de aprendizado de 0,001 e *batch size* de 32. Em cada rodada, os clientes executam 3 épocas de treinamento local antes de reportar as atualizações ao servidor. O código completo e os scripts de reprodução estão disponíveis no GitHub¹.

3.1. Protocolo de avaliação

Ao final de cada rodada de treinamento, o servidor transmite os K modelos globais atualizados para os clientes selecionados. Cada nó avalia individualmente o desempenho de todo o conjunto $\mathcal{W}_t$ em seu conjunto de teste local e identifica o modelo que apresenta a menor perda empírica. Uma vez identificado o melhor candidato, o cliente reporta a acurácia e a perda deste modelo específico ao servidor. As métricas globais são então obtidas através da média ponderada dessas respostas.

3.2. Expectativas

No cenário de seleção mínima ($\rho = 0$ e $N = 1$), antecipa-se que ambos os algoritmos apresentem desempenhos similares, uma vez que ambos convergem para um regime de agrupamento disjunto, selecionando apenas o modelo de melhor desempenho local. Contudo, a hipótese central deste trabalho é que o FLASC apresentará vantagens significativas nos demais cenários por dois motivos principais:

1. **Adaptabilidade da seleção:** No cenário intermediário ($\rho = 0,5$), enquanto o FLSC é forçado a incluir exatamente $N = 2$ modelos, o FLASC sabe determinar quantos modelos possuem uma perda aceitável para integrar a fusão baseado em ρ . Espera-se que isso evite modelos com desempenho ruim de degradarem o treinamento local e a agregação global.
2. **Fusão ponderada:** No cenário de seleção máxima ($\rho = 1$ e $N = 4$), onde todos os modelos são incluídos, o diferencial do FLASC reside na atribuição de pesos $\alpha_{c,i}$ (Linha 14 do Algoritmo 1). Enquanto o FLSC realiza uma média simples dos modelos, o FLASC prioriza aqueles com menor erro local. Espera-se que essa ponderação resulte em uma convergência mais rápida e estável, mesmo em configurações altamente inclusivas.

3.3. Reprodutibilidade

Devido à natureza estocástica inerente ao aprendizado federado, foram adotadas medidas para assegurar a reprodutibilidade dos experimentos. O controle de aleatoriedade foi implementado por meio da fixação de sementes em componentes críticos, incluindo no particionamento dos dados entre os clientes, na inicialização dos pesos das redes e nas sub-rotinas de treinamento e avaliação. Cada cenário descrito na Tabela 1 foi submetido a cinco execuções independentes, utilizando o conjunto de sementes $\{42, 43, 44, 45, 46\}$.

¹<https://github.com/nogueirapv/SimuAprendizadoFederado>

Referências

- [1] Brendan McMahan, Ewan Moore, Daniel Ramage, Seth Hampson, and Blaise Agueray Arcas. Communication-efficient learning of deep networks from decentralized data. In *Proceedings of the 20th International Conference on Artificial Intelligence and Statistics (AISTATS)*, 2017.
- [2] Chengxi Li, Gang Li, and Pramod K. Varshney. Federated learning with soft clustering. *IEEE Internet of Things Journal*, 9(10):7773–7782, 2022.
- [3] Christopher Briggs, Zhong Fan, and Peter Andras. Federated learning with hierarchical clustering of local updates to improve training on non-iid data. In *2020 International Joint Conference on Neural Networks (IJCNN)*, pages 1–9. IEEE, 2020.
- [4] Avishek Ghosh, Jichan Chung, Dong Yin, and Kannan Ramchandran. An efficient framework for clustered federated learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 33, pages 19586–19597, 2020.
- [5] Alex Krizhevsky et al. Learning multiple layers of features from tiny images. 2009.
- [6] Tzu-Ming Harry Hsu, Hang Qi, and Matthew Brown. Measuring the effects of non-identical data distributions for federated learning. *arXiv preprint arXiv:1909.06335*, 2019.
- [7] Daniel J Beutel, Taner Topal, Akhil Mathur, Diego Scheidegger, and Nicholas D Lane. Flower: A friendly federated learning research framework. *arXiv preprint arXiv:2007.14390*, 2020.
- [8] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, et al. Pytorch: An imperative style, high-performance deep learning library. In *Advances in Neural Information Processing Systems*, pages 8024–8035, 2019.